

GetGuac — Aggressive / Adversarial Test Plan

For experienced testers and pre-launch hardening. Every test here is designed to **break** something. If a test "passes" (the app handles it gracefully) — great. If it crashes, leaks data, accepts garbage, or silently fails — file a bug.

Do NOT run this against a production account with real data. Make a throwaway test account first.

Last updated: v0.3.0 (2026-05-29) · **Estimated time to run end-to-end:** 2-3 hours

Before you start

You'll need:

- A throwaway tester account (use the Tester Data Importer at `/admin` to populate it; the **Clear all test data** button cleans up between runs).
- Chrome **DevTools** familiarity (Network tab + Console tab).
- The TEST_DATA_1000.csv to load known state when needed.
- Optional but useful: a VPN to test from different regions, an Android emulator alongside a real phone.

For each test below:

1. Do the action.
 2. Watch DevTools Console + Network tab.
 3. Record:
 - Was there a JavaScript error in the console?
 - Did the server respond with 5xx?
 - Did the UI silently swallow the failure?
 - Did your data look corrupt afterwards?
-

Section 1 — Auth surface attacks

1.1 — Injection in username

On `/register`, try each of these as the username (use a unique email each time):

Input	Expected
' OR 1=1 --	Username invalid (only alphanumeric + . _ - allowed)
<script>alert(1)</script>	Username invalid
admin' --	Username invalid
\n\r\t	Username invalid
aa (100 chars)	Username invalid (max 32)
🥑 avocado	Username invalid
ADMIN	Should be normalized to lowercase OR rejected

 **Bug if:** any of these are accepted as a username.

1.2 — Injection in first/last name

Some fields are NOT username-validated. Try these as first name:

Input	Expected
<code><script>alert('xss')</script></code>	Stored but rendered as text (no script execution) anywhere it's displayed
<code>' ; DROP TABLE profiles; --</code>	Stored safely, no DB damage
Very long string (5000 chars)	Either truncated or rejected; no server 500
<code>\x00 \x01 \x02</code> (control chars)	Stored or stripped, no crash

After signup, check the dashboard, receipt header (your name appears in greetings), and any "Your account" pages. If you ever see `<script>` text but JS executes anyway → **critical XSS**.

1.3 — Email parsing edge cases

Input	Expected
<code>test@10minutemail.com</code>	Rejected: "Please use a permanent email"
<code>test@mailinator.com</code>	Rejected: disposable
<code>test@gmail.com</code> (double @)	Rejected: invalid email
<code>test@gétguac.com</code> (unicode lookalike)	Either accepted (and noted as different from getguac.com) or rejected
<code>Test@Gmail.com</code> (mixed case)	Normalized to lowercase; same user as <code>test@gmail.com</code> (no duplicate accounts)
<code>test+spam@gmail.com</code> (plus-addressing)	Accepted
<code><script>@a.com</code>	Rejected
Empty email	Rejected
320-char email (max RFC length)	Either accepted or clean rejection

1.4 — Rate-limit bypass attempts

1. Submit `/register` 6 times in 60 seconds with different emails.
2. Wait for the rate limit.
3. Try to bypass by:

- Changing your User-Agent header (DevTools → Network → re-issue request)
- Sending via a different IP (use a VPN, switch IPs)
- Submitting via DevTools → Console with `fetch('/api/auth/sign-up', ...)` directly

✅ **Expected:** Per-IP rate limit holds. VPN switch creates a new bucket (expected). Direct fetch still rate-limited.

🐛 **Bug if:** changing User-Agent alone bypasses the limit.

1.5 — Concurrent signup race

Open 5 incognito tabs. Type the same username + a different email in each. Click submit on all 5 within ~1 second.

✅ **Expected:** Exactly ONE succeeds. Others get "username taken" or "race conflict".

🐛 **Bug if:** two accounts get created with the same username.

1.6 — Honeypot trap

Open DevTools Console on `/register`. Run:

```
document.querySelector('input[name="website"]').value = 'http://spam.example.com'
```

Then submit the form normally with valid data.

✅ **Expected:** Server returns 400 "Invalid request" — no account created.

🐛 **Bug if:** account gets created anyway.

1.7 — Turnstile bypass

Open DevTools → Network → submit a registration form. In the POST body for `/api/auth/sign-up`, manually edit `turnstile_token` to `''` (empty) or `'fake-token-12345'` and replay.

✅ **Expected:** If `TURNSTILE_SECRET_KEY` is configured in production, server rejects with "CAPTCHA verification failed". If not configured (local dev), silently accepts (acceptable behavior — Turnstile is opt-in).

🐛 **Bug if:** `turnstile_token` is checked but a bad token is accepted on production.

Section 2 — Receipt parser stress

2.1 — Image that isn't a receipt

Upload (via web `/receipts` or mobile camera):

Image	Expected
A landscape photo (no text)	Either "couldn't parse" toast OR a low-confidence receipt the user can edit / delete
A screenshot of a webpage	Same
A blank white image	Either rejected or empty-content receipt
A 50 MB high-res photo	Either accepted with progress indicator OR clean "too large" message
An image with no extension (<code>.bin</code>)	Either format error OR accepted
An animated GIF	Either rejected OR first frame processed
A PDF disguised as <code>.jpg</code> (rename a PDF)	Clean error, no crash

 **Bug if:** any of these crash the parser OR silently fail without a toast.

2.2 — Receipt with adversarial content

Use a real receipt with these conditions:

Condition	Expected
Upside-down photo (180° rotated)	Either parsed correctly (Gemini can rotate) OR a "couldn't parse" message — not silent failure
Severely blurry photo	"Low confidence" warning OR clean reject
Receipt cut off (total off-screen)	Parses what's visible; flags total as \$0 or missing
Two receipts in one frame	Either picks one OR flags ambiguity
Non-Latin script (Hindi / Chinese receipt)	Either parses correctly OR clear "unsupported language"
Receipt with very low contrast (faded paper)	Best-effort parse with low confidence warning

2.3 — Bulk-upload abuse

Upload 50 receipts at once via web.

✅ **Expected:** Queued, processed in batches; clear progress indicator; nothing in the UI freezes.

🐛 **Bug if:** browser hangs, requests time out, or some receipts get lost.

2.4 — Duplicate detection

Upload the same receipt photo twice (exact same file).

✅ **Expected:** Either a "this looks like a duplicate of X" warning OR both get accepted (less ideal but not broken).

🐛 **Bug if:** the dedup logic crashes or silently merges wrong receipts.

Section 3 — Share security (post-migration 059)

3.1 — Token enumeration

In an incognito browser (no auth), visit:

```
https://getguac.app/share/aaaaaaaa
https://getguac.app/share/12345678
https://getguac.app/share/zzzzzzzz
```

✓ **Expected:** 404 Not Found page each time — NOT a 500, NOT a partial render with empty fields.

🐛 **Bug if:** any guessed token returns a real share (very unlikely but possible if RLS misconfigured).

3.2 — Expired token

Edit a real `shared_items` row in Supabase SQL Editor to have `expires_at = now() - interval '1 day'`. Then visit `/share/<that-token>` in incognito.

✓ **Expected:** 404. The `get_share_by_token` RPC filters expired rows.

3.3 — Direct table access via anon key

Open DevTools Console while NOT logged in (on `/login` page). Run:

```
const sb = window.supabase || (await import('@supabase/supabase-js')).createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY,
)
// Try direct table read
const { data, error } = await sb.from('shared_items').select('*').limit(5)
console.log({ data, error })
```

✓ **Expected:** `data` is `null` or empty, `error` mentions RLS / no policy. The migration 059 SELECT policy was REMOVED.

🐛 **Bug if:** `data` returns rows — migration 059 wasn't applied.

3.4 — Tamper with a share's payload

In DevTools (NOT logged in), try to UPDATE a known token:

```
await sb.from('shared_items')
  .update({ payload: { hacked: true }, expires_at: '2099-01-01' })
  .eq('token', 'KNOWN_TOKEN')
```

✓ **Expected:** rejected. No UPDATE policy for anon post-059.

🐛 **Bug if:** the update succeeds.

3.5 — View-count flood

Visit the same `/share/<token>` URL 1000 times in a loop:

```
for (let i = 0; i < 1000; i++) {  
  fetch('https://getguac.app/share/MYTOKEN', { headers: { Cache: 'no-cache' } })  
}
```

Check `shared_items.view_count` in Supabase before + after.

✅ **Expected:** `view_count` reaches ~1000 (atomic increments via RPC). May be slightly off (concurrent visitors), but in the hundreds at least.

🐛 **Bug if:** `view_count` stays low or stops incrementing — means the RPC failed silently.

3.6 — Replay attack on `/api/share/create`

Capture a successful share-create request from DevTools → Network. Replay it 50 times in 30 seconds via the Console.

✅ **Expected:** Rate-limited after ~30 attempts.

🐛 **Bug if:** all 50 succeed AND you accumulate 50 different valid tokens for the same payload (no dedup is expected, but verify limits).

Section 4 — Smashlist + Stash boundary cases

4.1 — Massive Smashlist

1. Add 1000 items manually to Your Smashlist (script via DevTools Console or repeated importer runs).
2. Open `/shopping` and scroll.

✅ **Expected:** Page loads, scroll is smooth, accordion still works.

🐛 **Bug if:** page freezes, infinite scroll breaks, or filter chips disappear.

4.2 — Auto-Add with no store history

Create 5 Smashlist items that have NEVER appeared on any of your receipts (e.g., "ZIGGURAT_TEST_ITEM_1" through 5). Open Buy Again somehow (force via DB if needed), tick the checkboxes, hit the "Send to" dropdown.

✅ **Expected:** Dropdown shows "No matching store history" + is disabled.

4.3 — Bulk select + bulk delete 50 items

Select 50+ Smashlist items, click Delete.

✅ **Expected:** Confirm dialog says "Delete 50 items". Click yes → all gone with one toast.

🐛 **Bug if:** It deletes one at a time with 50 toasts.

4.4 — Toggle predicted ↔ approved rapidly

Click the trash icon on a Predicted item 20 times in 5 seconds (toggles approved=false → back into Buy Again → click checkbox → back to approved → trash again ...).

✅ **Expected:** State stays consistent. No duplicate rows. No "lost" state.

🐛 **Bug if:** the item shows in both Buy Again AND Your Smashlist at the same time.

4.5 — Stash rater spam

Rate the same product 100 times in 30 seconds via DevTools Console:

```
for (let i = 1; i <= 100; i++) {  
  await fetch('/api/admin/import-test-data', /* won't work but rate test */);  
  // Or click each star 100 times manually  
}
```

✅ **Expected:** Optimistic updates land instantly; underlying DB writes happen via Promise.allSettled per store; final state is consistent.

🐛 **Bug if:** rating becomes inconsistent (some receipt_items show rating=3, others rating=5 for same product).

4.6 — On-hand stepper negative

Open DevTools, intercept the Supabase upsert call for `stash_inventory`, change `on_hand_qty` to `-5`.

✅ **Expected:** RLS rejects the row (`with check (on_hand_qty >= 0)` from migration 058).

🐛 **Bug if:** -5 lands in the DB.

4.7 — On-hand stepper absurdly high

Same as above but try `999999`.

✅ **Expected:** RLS rejects (`with check (on_hand_qty <= 9999)`).

4.8 — Compare Stores with 50 stores

Force a single product to have receipt history at 50 different stores. Open Compare Stores.

✅ **Expected:** Renders all 50 in a scrollable panel. Best-price callout picks the cheapest.

🐛 **Bug if:** the panel breaks layout or doesn't scroll.

Section 5 — GuacMoney + GuacScore boundary

5.1 — Direct write to `guac_money_events`

While signed in (NOT as admin), via DevTools:

```
await sb.from('guac_money_events').insert({
  user_id: '<some-other-user-id>',
  source: 'auto_add_cheapest',
  amount: 1000000,
})
```

✅ **Expected:** RLS rejects (`with check (user_id = auth.uid())`).

🐛 **Bug if:** the row lands.

5.2 — Negative GuacMoney amount

```
await sb.from('guac_money_events').insert({
  user_id: (await sb.auth.getUser()).data.user.id,
  source: 'auto_add_cheapest',
  amount: -500,
})
```

✓ **Expected:** RLS rejects (with check (amount > 0)).

5.3 — GuacMoney \$10,000+ single event

```
await sb.from('guac_money_events').insert({ user_id: ..., source: 'auto_add_cheapest'
```

✓ **Expected:** RLS rejects (with check (amount < 10000)).

5.4 — GuacScore at 100% ratings = harmful

Rate every product as 5 stars. Refresh dashboard.

✓ **Expected:** GuacScore tile reads " 🍌 Smash Master" with score near 100.

5.5 — GuacScore at 0% (every product 1 star)

✓ **Expected:** GuacScore reads " 🌱 Just Starting" — friendly tone (per [feedback memory: avoid competitor vocab](#)). NOT "Mushy" / "Lots of regret" (old label, killed in commit 062107b).

🐛 **Bug if:** the label reads anything demoralizing.

Section 6 — Performance under load

6.1 — 5000 receipts user

Import TEST_DATA_1000.csv 5 times into the same account.

Page	Expected load time
/dashboard	< 3 seconds
/receipts (paginated)	< 2 seconds per page
/shopping	< 3 seconds (predictor pre-computed)
/stash	< 4 seconds (aggregation across all items)

 **Bug if:** any page takes > 10 seconds or browser appears frozen.

6.2 — Concurrent dashboard refreshes

Open the dashboard in 5 tabs. Hit refresh on all simultaneously.

✓ **Expected:** All 5 load. No 429 rate limits. No data inconsistency between tabs.

6.3 — Bank statement parse for a 10-page PDF

Upload one of the larger generated statements (`statement-2026-05.pdf`).

✓ **Expected:** Parse completes within 30 seconds; transactions appear; interest charge categorized as `bank-fees` .

 **Bug if:** parse hangs > 60s OR misses obvious purchases.

Section 7 — Network resilience

7.1 — Offline mid-Smashlist-add

DevTools → Network → set throttling to **Offline**. Click Add on a Buy Again card.

✓ **Expected:** Clear error toast. No silent failure. Item is NOT marked as approved when you go back online.

 **Bug if:** the optimistic update sticks and there's no rollback.

7.2 — Slow 3G full app

DevTools → Network → "Slow 3G". Navigate /dashboard → /receipts → /shopping → /stash.

✓ **Expected:** Loading states render appropriately. Every page eventually loads. No 30-second blank screens.

7.3 — Server 500 from `/api/share/create`

DevTools → Network → block `/api/share/create`. Try to share an item.

✓ **Expected:** Clear "Share failed" toast. UI returns to normal state.

7.4 — Mid-upload disconnect

Start uploading a receipt photo. Pull the network cable / turn off WiFi mid-upload.

✓ **Expected:** Clear error after timeout. The half-uploaded receipt is NOT silently saved.

Section 8 — Mobile-specific hostile inputs

8.1 — Permission denial

Open the app cold. Tap the camera FAB. **Deny** camera permission.

✓ **Expected:** Clear message: "Camera access required. Open Settings to grant."

🐛 **Bug if:** the app crashes or silently fails.

8.2 — Background mid-parse

Start parsing a receipt. Hit the home button immediately.

Wait 60 seconds. Reopen the app.

✓ **Expected:** Parse either completes (if Gemini was already called) or fails cleanly with a re-try CTA.

8.3 — Force-kill mid-signup

On the signup screen, fill out the form, hit Submit, then **force-kill the app** before the response arrives.

Reopen → log in with those credentials.

✓ **Expected:** Either the account was created (you log in successfully) OR it wasn't (you see "user doesn't exist", can re-register). **NOT a half-created state.**

8.4 — Airplane mode mid-action

Toggle airplane mode while:

- A receipt is mid-upload
- A Buy Again "Add" is in flight
- Dashboard data is loading

✓ **Expected:** Each shows a clean error. Re-enabling network lets you retry.

8.5 — Out-of-storage mid-photo

Fill the phone's storage to 99% (large video file). Try to take a receipt photo.

✓ **Expected:** Clean "out of storage" error from the camera. App doesn't crash.

8.6 — Old Android version

If you have an old Android phone (Android 8 / API 26), try installing the v0.3.0 APK.

✓ **Expected:** Either installs and runs, OR a clear "requires Android X+" message.

Section 9 — Browser quirks

9.1 — Browser back during signup

Submit `/register`. While on the "Check your inbox" screen, hit browser **back**.

✓ **Expected:** Returns to the register form. The form is either pre-filled or empty (both acceptable). NO duplicate account on resubmit.

9.2 — Two tabs of `/shopping`

Open `/shopping` in two tabs. Add an item in tab 1. Switch to tab 2 and add a different item.

✓ **Expected:** Both items eventually appear in both tabs (after natural refresh interval or on next interaction).

🐛 **Bug if:** the two tabs diverge permanently OR one tab's writes overwrite the other's.

9.3 — Browser private/incognito

Run through Section 2.1 of TEST_PLAN.pdf in private/incognito mode.

✅ **Expected:** Identical experience. No "third-party cookie blocked" errors. No degraded auth flow.

9.4 — Tab close mid-form

Fill out the receipt-edit form on `/receipts/<id>`. Close the tab without saving.

Reopen. The form should be **empty** (no auto-saved draft), and the receipt should be unchanged.

🐛 **Bug if:** unsaved changes silently persist.

9.5 — DevTools Console errors

Open DevTools → Console. Navigate to:

- `/`
- `/dashboard`
- `/shopping`
- `/stash`
- `/receipts`
- `/share/preview`

✅ **Expected:** Zero red errors on each page. Yellow warnings are OK if they're third-party (Next.js dev hints, etc.).

🐛 **Bug if:** any page has a red error.

Section 10 — i18n / locale

10.1 — Non-English device locale (mobile)

Change device locale to **Hindi** or **Spanish** in Android Settings. Reopen GetGuac.

✅ **Expected:**

- Region flag in the app bar shows **IN** (until we ship the US-pinned override — verify it stays us US now).

- UI text stays in English (we haven't shipped i18n yet — this is correct as of v0.3.0).
- Numbers + dates use English format (we haven't localized either yet).

 **Bug if:** the app crashes OR text breaks layout.

10.2 — Web from non-US IP (VPN test)

Connect to a VPN exit in India / UK / Singapore. Visit `getguac.app/dashboard`.

✓ **Expected:** Region chip shows us US (per the pinned override — code change to flip is documented).

10.3 — Decimal/number formatting

Some locales use `1.000,50` (European) vs `1,000.50` (US). Try entering a price that way on /add-receipt manually.

✓ **Expected:** Clean parse OR clean reject. NO silent loss of decimals.

10.4 — RTL languages

If your browser supports it, force RTL via DevTools (`document.dir = 'rtl'`).

✓ **Expected:** Layout doesn't completely break. (We don't claim RTL support, but it shouldn't make the app unusable for someone whose language is RTL by default.)

Section 11 — DB consistency checks

For a power user with database access via Supabase SQL Editor. Each query should return **zero rows** on a healthy account.

11.1 — Orphaned receipt_items

```
select count(*)
from public.receipt_items ri
left join public.receipts r on r.id = ri.receipt_id
where r.id is null;
```

 **Bug if:** non-zero (FK constraint was bypassed somehow).

11.2 — Receipts with no items AND no total

```
select id, store_name, date
from public.receipts r
where total_amount = 0
    and not exists (select 1 from public.receipt_items where receipt_id = r.id)
    and r.from_statement = false;
```

A few may be legitimate (cash receipts), but a huge count means parse failures aren't getting cleaned up.

11.3 — Shopping list rows referencing deleted stores

```
select sl.id, sl.item_name, sl.store_name_id
from public.shopping_list sl
where sl.store_name_id is not null
    and not exists (select 1 from public.stores where id::text = sl.store_name_id);
```

 **Bug if:** non-zero — dangling FK.

11.4 — GuacMoney events without a corresponding cheapest-add

```
select e.id, e.amount, e.item_name
from public.guac_money_events e
where e.source = 'auto_add_cheapest'
    and e.amount > 100;
```

Any single event > \$100 from cheapest routing is suspicious — sanity-check the inputs.

Section 12 — Sign-off

Send back:

1. **Critical bugs first** — anything that:

- Crashes / 500s

- Leaks data
 - Allows actions you shouldn't be able to do
 - Loses your data
2. **Major bugs second** — UX broken, feature doesn't work end-to-end, performance unacceptable.
 3. **Cosmetic third** — visual glitches, copy issues, alignment problems.
 4. **Sections you couldn't complete** — and why.
 5. **Anything that felt OFF but you couldn't pin down** — vibes-based feedback is welcome.

Thanks for stress-testing! 🥑